

A Topologia do Sofrimento Psíquico: Calibração de Instrumentos Psicométricos via Inteligência Artificial Explicável

Frederico Pedrosa

2026-07-24

Contents

1	Set Up	1
2	Linhas de base: matrizes e modelos de mensuração empíricos	2
3	Algoritmos preditivos	7
3.1	Validação preditiva a nível de item isolado	12
4	Matrizes Sintéticas, Pseudo-CFA e Alinhamento Estrutural	14
5	XAI: Árvore de Inferência e Importância de Features	18
5.1	Plotagem da Importância de Features (XAI)	19

1 Set Up

```
library(tidyverse)
library(psych)
library(lavaan)
library(semTools)
library(corr)
library(tidyverse)
library(caret)
library(ranger)
library(partykit)
library(glmnet)
library(xgboost)
library(partykit)
library(ggparty)
library(ggplot2)
library(embedR)
library(patchwork)
library(dplyr)
library(randomForest)
```

2 Linhas de base: matrizes e modelos de mensuração empíricos

```
set.seed(42) # Reprodutibilidade global
options(mc.cores = parallel::detectCores() - 1)

# 1. CARREGAMENTO E LIMPEZA DOS DADOS -----
cat("[INFO] Carregando dados originais...\n")
```

```
## [INFO] Carregando dados originais...
```

```
# Lembre-se de ajustar os caminhos para o seu diretório
dado_dass <- readRDS("dado_dass.rds")
data_mapfre <- read.csv("dataset_original_mapfre.csv")
load("~/IEAT/dados/data.RData")

# Extração DASS-21
DASS21_itens <- dado_dass %>% select(64:84) %>% drop_na()

# Extração BDI-II e BAI (Conjunto e Isolados)
BDI_BAI_itens <- data_mapfre %>%
  select(starts_with("bdi_"), starts_with("bai_")) %>%
  rename_with(~ gsub("_", "", .)) %>%
  drop_na()

bdi_itens_df <- BDI_BAI_itens %>% select(starts_with("bdi"))
bai_itens_df <- BDI_BAI_itens %>% select(starts_with("bai"))

# Extração PANAS (Completa e Apenas Negativos)
PANAS_itens_full <- data[, 97:116] %>% drop_na()
itens_panas_neg <- c("PN2envergo", "PN4aflit", "PN6culpado", "PN8irrit",
  "PN10medo", "PN12hostil", "PN14inque", "PN16nervo",
  "PN18apavo", "PN20chate")
PANAS_neg_df <- PANAS_itens_full %>% select(any_of(itens_panas_neg))

# 2. MATRIZES POLICÓRICAS (ALVOS PARA A IA) -----
cat("[INFO] Calculando Matrizes Policóricas Humanas...\n")
```

```
## [INFO] Calculando Matrizes Policóricas Humanas...
```

```
extrair_alvo_poly <- function(df_itens) {
  psych::polychoric(df_itens)$rho %>%
    as_cordf() %>%
    stretch(remove.dups = TRUE, na.rm = TRUE) %>%
    rename(r_real = r, Var1 = x, Var2 = y)
}

df_poly_dass21 <- extrair_alvo_poly(DASS21_itens)
df_poly_bdibai <- extrair_alvo_poly(BDI_BAI_itens)
df_poly_bdi <- extrair_alvo_poly(bdi_itens_df)
df_poly_bai <- extrair_alvo_poly(bai_itens_df)
df_poly_panas_neg <- extrair_alvo_poly(PANAS_neg_df)
```

```

df_poly_panas      <- extrair_alvo_poly(PANAS_itens_full)

# 3. SINTAXE DOS 8 MODELOS TEÓRICOS (LAVAN) -----

# -- DASS-21 --
mod_dass21_3f <- '
  Depressao =~ i3 + i5 + i10 + i13 + i16 + i17 + i21
  Ansiedade =~ i2 + i4 + i7 + i9 + i15 + i19 + i20
  Estresse  =~ i1 + i6 + i8 + i11 + i12 + i14 + i18
'

mod_dass21_bf <- paste0("FG =~ ", paste0("i", 1:21, collapse = " + "), "\n", mod_dass21_3f)

# -- PANAS --
mod_panas_2f <- '
  Positivo =~ PN1ativo + PN3atento + PN5determ + PN7empol + PN9interes + PN11orgul + PN13alerta + PN15er
  Negativo =~ PN2envergo + PN4aflit + PN6culpado + PN8irrit + PN10medo + PN12hostil + PN14inquire + PN16
  Negativo =~ PN13alerta
'

mod_panas_neg <- '
  Negativo =~ PN2envergo + PN4aflit + PN6culpado + PN8irrit + PN10medo + PN12hostil + PN14inquire + PN16
'

# -- BDI-II e BAI --
mod_bdi_isolado <- "
  BDI_Cogn =~ bdi1 + bdi2 + bdi3 + bdi4 + bdi5 + bdi6 + bdi7 + bdi8 + bdi9 + bdi10 + bdi11 + bdi12 + bdi13 + bdi14 + bdi15 + bdi16 + bdi17 + bdi18 + bdi19 + bdi20 + bdi21
  BDI_Soma =~ bdi14 + bdi15 + bdi16 + bdi17 + bdi18 + bdi19 + bdi20 + bdi21
"

mod_bai_isolado <- "
  BAI_Cogn =~ bai4 + bai5 + bai8 + bai9 + bai10 + bai11 + bai14 + bai16 + bai17
  BAI_Soma =~ bai1 + bai2 + bai3 + bai6 + bai7 + bai12 + bai13 + bai15 + bai18 + bai19 + bai20 + bai21
"

mod_bdibai_4f <- paste0(mod_bdi_isolado, "\n", mod_bai_isolado)
mod_bdibai_bf <- paste0("FG =~ ", paste0("bdi", 1:21, collapse = " + "), " + ", paste0("bai", 1:21, collapse = " + "), "\n", mod_bdibai_4f)

# 4. AJUSTE DOS MODELOS (WLSMV E ML) -----
cat("[INFO] Executando CFAs (WLSMV e ML)... Isso pode levar alguns segundos...\n")

```

```

## [INFO] Executando CFAs (WLSMV e ML)... Isso pode levar alguns segundos...

```

```

extrair_metricas <- function(fit, nome_modelo) {
  m <- fitMeasures(fit, c("chisq", "df", "pvalue", "cfi", "tli", "rmsea", "rmsea.ci.lower", "rmsea.ci.upper", "srmr"))

  data.frame(
    Modelo = nome_modelo,
    ChiSq_df = round(m["chisq"] / m["df"], 2),
    p_value  = ifelse(m["pvalue"] < .001, "<.001", as.character(round(m["pvalue"], 3))),
    CFI      = round(m["cfi"], 3),
    TLI      = round(m["tli"], 3),
    RMSEA_IC = paste0(round(m["rmsea"], 3), " [", round(m["rmsea.ci.lower"], 3), " - ", round(m["rmsea.ci.upper"], 3), " ]"),
    SRMR     = round(m["srmr"], 3),
    stringsAsFactors = FALSE,
    row.names = NULL
  )
}

```

```

}

# --- 4.1 ESTIMADOR WLSMV (TABELA 1 - EMPÍRICA) ---
# Modelos de Fatores Correlacionados (Orthogonal = FALSE)
fit_dass_3f_w      <- cfa(mod_dass21_3f, data = DASS21_itens, ordered = TRUE, estimator = "WLSMV")
fit_panas_2f_w     <- cfa(mod_panas_2f, data = PANAS_itens_full, ordered = TRUE, estimator = "WLSMV")
fit_panas_neg_w    <- cfa(mod_panas_neg, data = PANAS_neg_df, ordered = TRUE, estimator = "WLSMV")
fit_bdi_w          <- cfa(mod_bdi_isolado, data = bdi_itens_df, ordered = TRUE, estimator = "WLSMV")
fit_bai_w          <- cfa(mod_bai_isolado, data = bai_itens_df, ordered = TRUE, estimator = "WLSMV")
fit_bdibai_4f_w    <- cfa(mod_bdibai_4f, data = BDI_BAI_itens, ordered = TRUE, estimator = "WLSMV")

# Modelos Bifatoriais (Orthogonal = TRUE)
fit_dass_bf_w      <- cfa(mod_dass21_bf, data = DASS21_itens, ordered = TRUE, estimator = "WLSMV", ortho
fit_bdibai_bf_w    <- cfa(mod_bdibai_bf, data = BDI_BAI_itens, ordered = TRUE, estimator = "WLSMV", ortho

Tabela_1_WLSMV <- bind_rows(
  extrair_metricas(fit_dass_3f_w, "DASS-21 com 3 Fatores"),
  extrair_metricas(fit_dass_bf_w, "DASS-21 Bifatorial (p-factor)"),
  extrair_metricas(fit_panas_2f_w, "PANAS 2 Fatores"),
  extrair_metricas(fit_panas_neg_w, "PANAS - Afetos Negativos"),
  extrair_metricas(fit_bai_w, "BAI 2 fatores"),
  extrair_metricas(fit_bdi_w, "BDI-II 2 Fatores"),
  extrair_metricas(fit_bdibai_4f_w, "BDI-II + BAI 2 com 4 Fatores"),
  extrair_metricas(fit_bdibai_bf_w, "BDI-II + BAI Bifatorial")
)

# --- 4.2 ESTIMADOR ML + PEARSON (TABELA S1 - MATERIAL SUPLEMENTAR) ---
cor_p_dass      <- cor(DASS21_itens, use = "pairwise.complete.obs")
cor_p_panas_f   <- cor(PANAS_itens_full, use = "pairwise.complete.obs")
cor_p_panas_n   <- cor(PANAS_neg_df, use = "pairwise.complete.obs")
cor_p_bdi       <- cor(bdi_itens_df, use = "pairwise.complete.obs")
cor_p_bai       <- cor(bai_itens_df, use = "pairwise.complete.obs")
cor_p_bdibai    <- cor(BDI_BAI_itens, use = "pairwise.complete.obs")

fit_dass_3f_ml   <- cfa(mod_dass21_3f, sample.cov = cor_p_dass, sample.nobs = nrow(DASS21_itens), estim
fit_dass_bf_ml   <- cfa(mod_dass21_bf, sample.cov = cor_p_dass, sample.nobs = nrow(DASS21_itens), estim
fit_panas_2f_ml  <- cfa(mod_panas_2f, sample.cov = cor_p_panas_f, sample.nobs = nrow(PANAS_itens_full),
fit_panas_neg_ml <- cfa(mod_panas_neg, sample.cov = cor_p_panas_n, sample.nobs = nrow(PANAS_neg_df), es
fit_bai_ml       <- cfa(mod_bai_isolado, sample.cov = cor_p_bai, sample.nobs = nrow(bai_itens_df), estim
fit_bdi_ml       <- cfa(mod_bdi_isolado, sample.cov = cor_p_bdi, sample.nobs = nrow(bdi_itens_df), estim
fit_bdibai_4f_ml <- cfa(mod_bdibai_4f, sample.cov = cor_p_bdibai, sample.nobs = nrow(BDI_BAI_itens), es
fit_bdibai_bf_ml <- cfa(mod_bdibai_bf, sample.cov = cor_p_bdibai, sample.nobs = nrow(BDI_BAI_itens), es

Tabela_S1_ML <- bind_rows(
  extrair_metricas(fit_dass_3f_ml, "DASS-21 (3 Fatores) - ML"),
  extrair_metricas(fit_dass_bf_ml, "DASS-21 (Bifatorial) - ML"),
  extrair_metricas(fit_panas_2f_ml, "PANAS (2 Fatores) - ML"),
  extrair_metricas(fit_panas_neg_ml, "PANAS (Afetos Negativos) - ML"),
  extrair_metricas(fit_bai_ml, "BAI (2 Fatores) - ML"),
  extrair_metricas(fit_bdi_ml, "BDI-II (2 Fatores) - ML"),
  extrair_metricas(fit_bdibai_4f_ml, "BDI-II + BAI (4 Fatores) - ML"),
  extrair_metricas(fit_bdibai_bf_ml, "BDI-II + BAI (Bifatorial) - ML")
)

```

```
# 5. EXIBIÇÃO DAS TABELAS NO CONSOLE -----
cat("\n=====\\n")
```

```
##
## =====
```

```
cat("      TABELA 1: Ajuste dos Modelos Humanos (Padrão WLSMV)      \\n")
```

```
##      TABELA 1: Ajuste dos Modelos Humanos (Padrão WLSMV)
```

```
cat("=====\\n")
```

```
## =====
```

```
print(Tabela_1_WLSMV)
```

```
##
##          Modelo ChiSq_df p_value   CFI   TLI
## 1      DASS-21 com 3 Fatores    1.63   <.001 0.995 0.995
## 2 DASS-21 Bifatorial (p-factor)    1.04   0.343 1.000 1.000
## 3          PANAS 2 Fatores    3.80   <.001 0.982 0.980
## 4      PANAS - Afetos Negativos    3.06   <.001 0.993 0.992
## 5          BAI 2 fatores    4.64   <.001 0.988 0.986
## 6          BDI-II 2 Fatores    5.05   <.001 0.982 0.980
## 7 BDI-II + BAI 2 com 4 Fatores    3.46   <.001 0.985 0.984
## 8      BDI-II + BAI Bifatorial    4.73   <.001 0.979 0.976
##          RMSEA_IC  SRMR
## 1 0.049 [0.039 - 0.059] 0.057
## 2  0.013 [0 - 0.031] 0.045
## 3 0.078 [0.072 - 0.085] 0.072
## 4 0.067 [0.053 - 0.082] 0.050
## 5 0.045 [0.042 - 0.048] 0.056
## 6  0.047 [0.044 - 0.05] 0.058
## 7 0.037 [0.035 - 0.038] 0.054
## 8 0.045 [0.044 - 0.047] 0.063
```

```
cat("\n=====\\n")
```

```
##
## =====
```

```
cat("      TABELA S1: Baseline Humano para IA (Pearson + ML)      \\n")
```

```
##      TABELA S1: Baseline Humano para IA (Pearson + ML)
```

```
cat("=====\\n")
```

```
## =====
```

```
print(Tabela_S1_ML)
```

```
##                               Modelo ChiSq_df p_value   CFI   TLI
## 1      DASS-21 (3 Fatores) - ML      2.29  <.001 0.920 0.910
## 2      DASS-21 (Bifatorial) - ML      1.90  <.001 0.950 0.937
## 3      PANAS (2 Fatores) - ML      4.14  <.001 0.882 0.866
## 4  PANAS (Afetos Negativos) - ML      4.90  <.001 0.930 0.910
## 5      BAI (2 Fatores) - ML     10.32  <.001 0.865 0.849
## 6      BDI-II (2 Fatores) - ML      7.77  <.001 0.879 0.865
## 7  BDI-II + BAI (4 Fatores) - ML      5.87  <.001 0.844 0.835
## 8  BDI-II + BAI (Bifatorial) - ML      6.14  <.001 0.842 0.825
##                               RMSEA_IC  SRMR
## 1 0.071 [0.062 - 0.079] 0.057
## 2 0.059 [0.049 - 0.069] 0.041
## 3 0.083 [0.076 - 0.089] 0.063
## 4 0.092 [0.079 - 0.106] 0.044
## 5 0.071 [0.068 - 0.074] 0.047
## 6 0.061 [0.058 - 0.064] 0.045
## 7 0.051 [0.05 - 0.053] 0.044
## 8 0.053 [0.051 - 0.054] 0.055
```

```
# 6. CÁLCULO DAS FIDEDIGNIDADES (ÔMEGA) PARA A COLUNA DA TABELA -----
# cat("\n--- Ômegas de McDonald ---\n")
print(round(semTools::compRelSEM(fit_dass_3f_w), 2))
```

```
## Depressao Ansiedade Estresse
##      0.93      0.90      0.90
```

```
print(round(semTools::compRelSEM(fit_dass_bf_w), 2))
```

```
##      FG Depressao Ansiedade Estresse
##      0.87      0.33      0.18      0.08
```

```
print(round(semTools::compRelSEM(fit_panas_2f_w), 2))
```

```
## Positivo Negativo
##      0.93      0.92
```

```
print(round(semTools::compRelSEM(fit_panas_neg_w), 2))
```

```
## Negativo
##      0.91
```

```
print(round(semTools::compRelSEM(fit_bai_w), 2))
```

```
## BAI_Cogn BAI_Soma
##      0.87      0.85
```

```
print(round(semTools::compRelSEM(fit_bdi_w), 2))
```

```
## BDI_Cogn BDI_Soma  
##      0.85      0.82
```

```
print(round(semTools::compRelSEM(fit_bdibai_4f_w), 2))
```

```
## BDI_Cogn BDI_Soma BAI_Cogn BAI_Soma  
##      0.85      0.82      0.87      0.85
```

```
print(round(semTools::compRelSEM(fit_bdibai_bf_w), 2))
```

```
##      FG BDI_Cogn BDI_Soma BAI_Cogn BAI_Soma  
##      0.87      0.19      0.20      0.16      0.25
```

3 Algoritmos preditivos

```
# Chave mestra: TRUE para treinar do zero, FALSE para ler os RDS salvos  
TREINAR_MODELOS <- FALSE
```

```
# =====  
# NOTA DE REPRODUTIBILIDADE:  
# A rotina completa e computacionalmente intensiva de treinamento destes modelos  
# preditivos (com validação cruzada LOOCV, tuning de hiperparâmetros, etc.)  
# encontra-se no script avulso 'Treinar_Modelos_Preditivos.R', disponível  
# no repositório deste projeto no GitHub. Neste documento principal, priorizamos  
# o carregamento dos modelos já ajustados para otimizar a renderização.  
# =====
```

```
# 1. FUNÇÃO CORE: ENGENHARIA DE FEATURES -----
```

```
gerar_features_ia <- function(df_pares_alvo, matriz_emb) {  
  itens_validos <- df_pares_alvo$Var1 %in% rownames(matriz_emb) &  
    df_pares_alvo$Var2 %in% rownames(matriz_emb)  
  df_pares <- df_pares_alvo[itens_validos, ]
```

```
  lista_features <- lapply(1:nrow(df_pares), function(i) {  
    v1 <- matriz_emb[df_pares$Var1[i], ]  
    v2 <- matriz_emb[df_pares$Var2[i], ]  
  
    diff_abs <- abs(v1 - v2)  
    prod_had <- v1 * v2  
    c(diff_abs, prod_had)  
  })
```

```
  df_final <- as.data.frame(do.call(rbind, lista_features))  
  colnames(df_final) <- paste0("V", 1:ncol(df_final))
```

```
  df_final$r_real <- df_pares$r_real  
  df_final$Var1 <- df_pares$Var1
```

```

df_final$Var2 <- df_pares$Var2

return(df_final)
}

# 2. CARREGAMENTO DOS EMBEDDINGS -----
emb_dass_cohere <- readRDS("emb_dass_cohere.rds")
emb_dass_mpnet <- readRDS("emb_dass_mpnet.rds")
emb_dass_openai <- readRDS("matriz_emb_openai_dass.rds")

emb_panas_cohere <- readRDS("emb_panas_cohere.rds")
emb_panas_mpnet <- readRDS("emb_panas_mpnet.rds")
emb_panas_openai <- readRDS("matriz_emb_openai_panas.rds")

emb_clin_cohere <- readRDS("emb_clinico_cohere.rds")
emb_clin_mpnet <- readRDS("emb_clinico_mpnet.rds")
emb_clin_openai <- readRDS("matriz_emb_clinico_openai.rds")

# 3. CONSTRUÇÃO DOS DATASETS -----
df_train_cohere <- gerar_features_ia(df_poly_dass21, emb_dass_cohere)
df_train_mpnet <- gerar_features_ia(df_poly_dass21, emb_dass_mpnet)
df_train_openai <- gerar_features_ia(df_poly_dass21, emb_dass_openai)

df_trans_bb_cohere <- gerar_features_ia(df_poly_bdibai, emb_clin_cohere)
df_trans_bb_mpnet <- gerar_features_ia(df_poly_bdibai, emb_clin_mpnet)
df_trans_bb_openai <- gerar_features_ia(df_poly_bdibai, emb_clin_openai)

itens_negativos <- c("PN2envergo", "PN4aflit", "PN6culpado", "PN8irrit",
                    "PN10medo", "PN12hostil", "PN14inquiie", "PN16nervo",
                    "PN18apavo", "PN20chate")

df_poly_panas_neg <- df_poly_panas %>%
  filter(Var1 %in% itens_negativos & Var2 %in% itens_negativos)

df_trans_panas_cohere <- gerar_features_ia(df_poly_panas_neg, emb_panas_cohere)
df_trans_panas_mpnet <- gerar_features_ia(df_poly_panas_neg, emb_panas_mpnet)
df_trans_panas_openai <- gerar_features_ia(df_poly_panas_neg, emb_panas_openai)

# 4. CARREGAMENTO DOS MODELOS -----
# Consulte o arquivo 'Treinar_Modelos_Preditivos.R' para o código de geração destes objetos
mod_en_cohere <- readRDS("modelo_en.rds")
mod_rf_cohere <- readRDS("modelo_rf_dass21_v1.rds")
mod_xgb_cohere <- readRDS("modelo_xgb_final.rds")
mod_ctree_cohere <- readRDS("modelo_ctree.rds")

mod_en_mpnet <- readRDS("modelo_en_mpnet.rds")
mod_rf_mpnet <- readRDS("modelo_rf_mpnet.rds")
mod_xgb_mpnet <- readRDS("modelo_xgb_mpnet.rds")
mod_ctree_mpnet <- readRDS("modelo_ctree_mpnet.rds")

mod_en_openai <- readRDS("modelos_final/modelo_en_openai_full.rds")
mod_rf_openai <- readRDS("modelos_final/modelo_rf_openai_full.rds")
mod_xgb_openai <- readRDS("modelos_final/modelo_xgb_openai_full.rds")

```



```

mod_ctree_openai <- readRDS("modelos_final/modelo_ctree_openai.rds")

# 5. FUNÇÕES DE CÁLCULO DE MÉTRICAS (R2, RMSE, MAE) -----
get_train_metrics <- function(modelo, df_train, is_ctree = FALSE) {
  if (is_ctree) {
    preds <- as.numeric(predict(modelo, newdata = df_train))
    real <- df_train$r_real
    return(c(R2 = cor(preds, real)^2, RMSE = sqrt(mean((preds - real)^2)), MAE = mean(abs(preds - real)))
  } else {
    res <- modelo$results
    best <- res[which.min(res$RMSE), ]
    if(nrow(best) == 0 || is.na(best$Rsquared[1])) {
      preds <- predict(modelo, newdata = df_train)
      real <- df_train$r_real
      return(c(R2 = cor(preds, real)^2, RMSE = sqrt(mean((preds - real)^2)), MAE = mean(abs(preds - real)))
    }
    return(c(R2 = best$Rsquared, RMSE = best$RMSE, MAE = best$MAE))
  }
}

avaliar_transferencia_completa <- function(modelo = NULL, df_trans, is_ctree = FALSE, preds_vector = NULL) {
  if (!is.null(preds_vector)) {
    preds <- preds_vector
  } else if (is_ctree) {
    preds <- as.numeric(predict(modelo, newdata = df_trans))
  } else {
    preds <- predict(modelo, newdata = df_trans)
  }

  real <- df_trans$r_real
  r2_val <- cor(preds, real, use = "pairwise.complete.obs")^2
  rmse_val <- sqrt(mean((preds - real)^2))
  mae_val <- mean(abs(preds - real))

  return(c(R2 = r2_val, RMSE = rmse_val, MAE = mae_val))
}

get_cosine_preds <- function(df_pares, matriz_emb) {
  apply(df_pares, 1, function(row) {
    v1 <- matriz_emb[row["Var1"], ]
    v2 <- matriz_emb[row["Var2"], ]
    sum(v1 * v2) / sqrt(sum(v1^2) * sum(v2^2))
  })
}

gerar_baseline <- function(arq_nome, df_dass, emb_dass, df_panas, emb_panas, df_bb, emb_bb) {
  preds_dass <- get_cosine_preds(df_dass, emb_dass)
  preds_panas <- get_cosine_preds(df_panas, emb_panas)
  preds_bb <- get_cosine_preds(df_bb, emb_bb)

  met_panas <- avaliar_transferencia_completa(preds_vector = preds_panas, df_trans = df_panas)
  met_bb <- avaliar_transferencia_completa(preds_vector = preds_bb, df_trans = df_bb)
}

```

```
data.frame(
  Arquitetura = arq_nome,
  Modelo      = "Linear (Cosseno)",
  R2_DASS     = sprintf("%.3f", cor(preds_dass, df_dass$r_real, method = "pearson")^2),
  RMSE_DASS   = "N/A",
  MAE_DASS    = "N/A",
  R2_PANAS    = sprintf("%.3f", met_panas["R2"]),
  RMSE_PANAS  = sprintf("%.3f", met_panas["RMSE"]),
  MAE_PANAS   = sprintf("%.3f", met_panas["MAE"]),
  R2_BDI_BAI  = sprintf("%.3f", met_bb["R2"]),
  RMSE_BDI_BAI = sprintf("%.3f", met_bb["RMSE"]),
  MAE_BDI_BAI = sprintf("%.3f", met_bb["MAE"]),
  stringsAsFactors = FALSE
)
}
```

6. MONTAGEM DA TABELA 2 -----

```
config_modelos <- list(
  list(arq = "Cohere", mod = "Elastic Net", obj = mod_en_cohere, df_tr = df_train_cohere, is_ct = 1),
  list(arq = "Cohere", mod = "ctree", obj = mod_ctree_cohere, df_tr = df_train_cohere, is_ct = 1),
  list(arq = "Cohere", mod = "Random Forest", obj = mod_rf_cohere, df_tr = df_train_cohere, is_ct = 1),
  list(arq = "Cohere", mod = "XGBoost", obj = mod_xgb_cohere, df_tr = df_train_cohere, is_ct = 1),
  list(arq = "MPNet", mod = "Elastic Net", obj = mod_en_mpnet, df_tr = df_train_mpnet, is_ct = 1),
  list(arq = "MPNet", mod = "ctree", obj = mod_ctree_mpnet, df_tr = df_train_mpnet, is_ct = 1),
  list(arq = "MPNet", mod = "Random Forest", obj = mod_rf_mpnet, df_tr = df_train_mpnet, is_ct = 1),
  list(arq = "MPNet", mod = "XGBoost", obj = mod_xgb_mpnet, df_tr = df_train_mpnet, is_ct = 1),
  list(arq = "OpenAI", mod = "Elastic Net", obj = mod_en_openai, df_tr = df_train_openai, is_ct = 1),
  list(arq = "OpenAI", mod = "ctree", obj = mod_ctree_openai, df_tr = df_train_openai, is_ct = 1),
  list(arq = "OpenAI", mod = "Random Forest", obj = mod_rf_openai, df_tr = df_train_openai, is_ct = 1),
  list(arq = "OpenAI", mod = "XGBoost", obj = mod_xgb_openai, df_tr = df_train_openai, is_ct = 1)
)
```

```
linhas_ml <- lapply(config_modelos, function(cfg) {
  metrics <- get_train_metrics(cfg$obj, cfg$df_tr, cfg$is_ct)

  if(cfg$arq == "Cohere") { df_p <- df_trans_panas_cohere; df_b <- df_trans_bb_cohere }
  if(cfg$arq == "MPNet") { df_p <- df_trans_panas_mpnet; df_b <- df_trans_bb_mpnet }
  if(cfg$arq == "OpenAI") { df_p <- df_trans_panas_openai; df_b <- df_trans_bb_openai }
})
```

```
met_p <- avaliar_transferencia_completa(cfg$obj, df_p, cfg$is_ct)
met_b <- avaliar_transferencia_completa(cfg$obj, df_b, cfg$is_ct)
```

```
data.frame(
  Arquitetura = cfg$arq,
  Modelo      = cfg$mod,
  R2_DASS     = sprintf("%.3f", metrics["R2"]),
  RMSE_DASS   = sprintf("%.3f", metrics["RMSE"]),
  MAE_DASS    = sprintf("%.3f", metrics["MAE"]),
  R2_PANAS    = sprintf("%.3f", met_p["R2"]),
  RMSE_PANAS  = sprintf("%.3f", met_p["RMSE"]),
  MAE_PANAS   = sprintf("%.3f", met_p["MAE"]),
  R2_BDI_BAI  = sprintf("%.3f", met_b["R2"]),
  RMSE_BDI_BAI = sprintf("%.3f", met_b["RMSE"]),
)
```

```

    MAE_BDI_BAI = sprintf("%.3f", met_b["MAE"]),
    stringsAsFactors = FALSE
  )
})

tabela_ml <- do.call(rbind, linhas_ml)

baseline_cohere <- gerar_baseline("Cohere", df_train_cohere, emb_dass_cohere, df_trans_panas_cohere, emb_dass_panas_cohere)
baseline_mpnet <- gerar_baseline("MPNet", df_train_mpnet, emb_dass_mpnet, df_trans_panas_mpnet, emb_dass_panas_mpnet)
baseline_openai <- gerar_baseline("OpenAI", df_train_openai, emb_dass_openai, df_trans_panas_openai, emb_dass_panas_openai)

tabela_2_final <- bind_rows(
  baseline_cohere, tabela_ml[tabela_ml$Arquitetura == "Cohere",],
  baseline_mpnet, tabela_ml[tabela_ml$Arquitetura == "MPNet",],
  baseline_openai, tabela_ml[tabela_ml$Arquitetura == "OpenAI",]
)
rownames(tabela_2_final) <- NULL

cat("\n=====

```

```
##
```

```
## =====
```

```
cat("          TABELA 2 FINAL: Desempenho Completo (R2, RMSE e MAE para todas as fatias)
```

```
##          TABELA 2 FINAL: Desempenho Completo (R2, RMSE e MAE para todas as fatias)
```

```
cat("=====
```

```
## =====
```

```
print(tabela_2_final)
```

```
##      Arquitetura      Modelo R2_DASS RMSE_DASS MAE_DASS R2_PANAS RMSE_PANAS
## 1      Cohere Linear (Cosseno)  0.137      N/A      N/A      0.020      0.173
## 2      Cohere Elastic Net      0.659      0.069      0.054      0.281      0.108
## 3      Cohere      ctree      0.649      0.069      0.053      0.250      0.094
## 4      Cohere Random Forest      0.693      0.073      0.058      0.118      0.100
## 5      Cohere      XGBoost      0.599      0.076      0.060      0.265      0.091
## 6      MPNet Linear (Cosseno)  0.309      N/A      N/A      0.000      0.217
## 7      MPNet Elastic Net      0.690      0.066      0.052      0.014      0.128
## 8      MPNet      ctree      0.600      0.074      0.058      0.013      0.118
## 9      MPNet Random Forest      0.674      0.076      0.060      0.003      0.109
## 10     MPNet      XGBoost      0.630      0.071      0.056      0.024      0.122
## 11     OpenAI Linear (Cosseno)  0.208      N/A      N/A      0.070      0.119
## 12     OpenAI Elastic Net      0.691      0.068      0.054      0.163      0.097
## 13     OpenAI      ctree      0.276      0.102      0.075      0.180      0.100
## 14     OpenAI Random Forest      0.652      0.079      0.061      0.053      0.103
## 15     OpenAI      XGBoost      0.617      0.073      0.057      0.006      0.119
##      MAE_PANAS R2_BDI_BAI RMSE_BDI_BAI MAE_BDI_BAI
## 1      0.143      0.072      0.272      0.249
```

## 2	0.086	0.104	0.157	0.133
## 3	0.077	0.000	0.197	0.159
## 4	0.083	0.057	0.173	0.148
## 5	0.074	0.015	0.188	0.160
## 6	0.191	0.275	0.135	0.106
## 7	0.106	0.197	0.155	0.130
## 8	0.093	0.043	0.188	0.155
## 9	0.091	0.155	0.159	0.137
## 10	0.103	0.076	0.161	0.135
## 11	0.096	0.198	0.121	0.092
## 12	0.080	0.091	0.147	0.124
## 13	0.083	0.003	0.188	0.156
## 14	0.087	0.105	0.163	0.140
## 15	0.103	0.068	0.153	0.128

```
cat("=====
```

```
## =====
```

3.1 Validação preditiva a nível de item isolado

```
cat("[INFO] Executando Regressão Regularizada a Nível de Item (LOOCV)...\n")
```

```
## [INFO] Executando Regressão Regularizada a Nível de Item (LOOCV)...
```

```
# 1. FUNÇÃO DE EXTRAÇÃO E TREINAMENTO LOOCV -----
rodar_loocv_item <- function(fit_obj, matriz_emb, nome_alvo, fator_especifico = NULL) {

  # Extrair cargas fatoriais empíricas humanas (Y)
  cargas <- standardizedSolution(fit_obj) %>% filter(op == "=~")

  if (!is.null(fator_especifico)) {
    cargas <- cargas %>% filter(lhs == fator_especifico)
  }

  y_target <- cargas$est.std
  names(y_target) <- cargas$rhs

  # Preparar a matriz preditora de embeddings (X)
  X_preds <- as.data.frame(matriz_emb[names(y_target), ])
  colnames(X_preds) <- paste0("V", 1:ncol(X_preds))

  # Configurar e Treinar o Elastic Net com LOOCV Estrito
  set.seed(42)
  ctrl_loocv <- trainControl(method = "LOOCV")

  modelo <- train(
    x = X_preds,
    y = y_target,
    method = "glmnet",
```

```

    trControl = ctrl_loocv
  )

  # Previsões e Métricas
  preds <- predict(modelo, X_preds)
  res_spearman <- cor.test(preds, y_target, method = "spearman")
  rmse_val <- sqrt(mean((preds - y_target)^2))
  mae_val <- mean(abs(preds - y_target))

  # Retornar linha formatada para a tabela
  data.frame(
    Instrumento = nome_alvo,
    N_Itens     = length(y_target),
    Spearman_r   = round(res_spearman$estimate, 3),
    p_valor      = signif(res_spearman$p.value, 3),
    RMSE         = round(rmse_val, 3),
    MAE          = round(mae_val, 3),
    stringsAsFactors = FALSE, row.names = NULL
  )
}

# 2. EXECUÇÃO PARA TODOS OS INSTRUMENTOS -----
linhas_loocv <- list()

# DASS-21 (Fator Geral do modelo Bifatorial)
linhas_loocv[[1]] <- rodar_loocv_item(fit_dass_bf_w, emb_dass_cohere, "DASS-21 (Fator Geral)", fator_es)

# BDI-II (2 Fatores)
linhas_loocv[[2]] <- rodar_loocv_item(fit_bdi_w, emb_clin_cohere, "BDI-II (Completo)")

# BAI (2 Fatores)
linhas_loocv[[3]] <- rodar_loocv_item(fit_bai_w, emb_clin_cohere, "BAI (Completo)")

# PANAS (Afetos Negativos)
linhas_loocv[[4]] <- rodar_loocv_item(fit_panas_neg_w, emb_panas_cohere, "PANAS (Afetos Negativos)")

# BDI-II + BAI (Fator Geral do modelo Bifatorial Unificado)
linhas_loocv[[5]] <- rodar_loocv_item(fit_bdibai_bf_w, emb_clin_cohere, "BDI-II + BAI (Fator Geral)", f)

# 3. CONSOLIDAÇÃO E EXIBIÇÃO DA TABELA -----
tabela_loocv_itens <- do.call(rbind, linhas_loocv)

cat("\n=====\\n")

##
## =====

cat("      TABELA 3: Validação Preditiva a Nível de Item (Zero-Shot)      \\n")

##      TABELA 3: Validação Preditiva a Nível de Item (Zero-Shot)

```

```
cat("=====\\n")
```

```
## =====
```

```
print(tabela_loocv_itens)
```

```
##           Instrumento N_Itens Spearman_r p_valor RMSE MAE
## 1      DASS-21 (Fator Geral)      21      0.986 4.49e-06 0.012 0.010
## 2      BDI-II (Completo)      21      0.990 4.35e-06 0.009 0.008
## 3      BAI (Completo)      21      0.742 1.82e-04 0.082 0.068
## 4  PANAS (Afetos Negativos)     10      0.855 3.50e-03 0.067 0.055
## 5 BDI-II + BAI (Fator Geral)    42      0.902 0.00e+00 0.062 0.047
```

```
cat("=====\\n")
```

```
## =====
```

4 Matrizes Sintéticas, Pseudo-CFA e Alinhamento Estrutural

```
cat("[INFO] Construindo Matrizes Sintéticas da IA e rodando Pseudo-CFAs...\\n")
```

```
## [INFO] Construindo Matrizes Sintéticas da IA e rodando Pseudo-CFAs...
```

```
# 1. FUNÇÕES AUXILIARES -----
```

```
gerar_features_pares <- function(comparacao_df, matriz_embeddings) {
  features_list <- list()
  for(i in 1:nrow(comparacao_df)) {
    v1 <- matriz_embeddings[comparacao_df$Var1[i], ]
    v2 <- matriz_embeddings[comparacao_df$Var2[i], ]
    features_list[[i]] <- c(abs(v1 - v2), v1 * v2)
  }
  df_final <- as.data.frame(do.call(rbind, features_list))
  colnames(df_final) <- paste0("V", 1:ncol(df_final))
  return(df_final)
}
```

```
gerar_features_pares_compativel <- function(df_pares, matriz_emb, modelo_ref) {
  vars_modelo <- names(modelo_ref$trainingData)[!(names(modelo_ref$trainingData) %in% c(".outcome", "Va
```

```
  lista_features <- lapply(1:nrow(df_pares), function(i) {
    v1 <- matriz_emb[df_pares$Var1[i], ]
    v2 <- matriz_emb[df_pares$Var2[i], ]
    c(abs(v1 - v2), (v1 * v2))
  })
```

```
  df_final <- as.data.frame(do.call(rbind, lista_features))
  n_dims <- ncol(matriz_emb)
  colnames(df_final) <- c(paste0("diff_", 1:n_dims), paste0("V", 1:n_dims))
```

```

missing_vars <- setdiff(vars_modelo, names(df_final))
if(length(missing_vars) > 0) {
  df_final[missing_vars] <- 0
}

return(df_final[, vars_modelo, drop = FALSE])
}

reconstruir_matriz <- function(df_pares, vetor_itens) {
  n <- length(vetor_itens)
  matriz <- matrix(1, nrow = n, ncol = n, dimnames = list(vetor_itens, vetor_itens))
  for(i in 1:nrow(df_pares)) {
    matriz[df_pares$Var1[i], df_pares$Var2[i]] <- df_pares$r_pred[i]
    matriz[df_pares$Var2[i], df_pares$Var1[i]] <- df_pares$r_pred[i]
  }
  return(matriz)
}

# 2. GERAÇÃO DAS MATRIZES SINTÉTICAS -----

# DASS-21
itens_dass <- paste0("i", 1:21)
df_pares_dass <- df_poly_dass21
df_features_dass <- gerar_features_pares(df_pares_dass, emb_dass_cohere)
df_pares_dass$r_pred <- predict(mod_en_cohere, newdata = df_features_dass)
matriz_sintetica_dass <- reconstruir_matriz(df_pares_dass, itens_dass)

# PANAS (Afetos Negativos)
# PANAS (Afetos Negativos) - Usando o modelo generalizado do Cohere
itens_panas_neg <- c("PN2envergo", "PN4aflit", "PN6culpado", "PN8irrit",
                    "PN10medo", "PN12hostil", "PN14inquire", "PN16nervo",
                    "PN18apavo", "PN20chate")

df_poly_panas_neg$r_pred <- predict(mod_en_cohere, newdata = gerar_features_pares(df_poly_panas_neg, emb_panas_cohere))
matriz_sintetica_panas <- reconstruir_matriz(df_poly_panas_neg, itens_panas_neg)

# BDI-II (Isolado)
itens_bdi <- paste0("bdi", 1:21)
df_features_bdi_indep <- gerar_features_pares_compativel(df_poly_bdi, emb_clin_cohere, mod_en_cohere)
df_poly_bdi$r_pred <- as.numeric(predict(mod_en_cohere, newdata = df_features_bdi_indep))
matriz_sintetica_bdi <- reconstruir_matriz(df_poly_bdi, itens_bdi)

# BAI (Isolada)
itens_bai <- paste0("bai", 1:21)
df_features_bai_indep <- gerar_features_pares_compativel(df_poly_bai, emb_clin_cohere, mod_en_cohere)
df_poly_bai$r_pred <- as.numeric(predict(mod_en_cohere, newdata = df_features_bai_indep))
matriz_sintetica_bai <- reconstruir_matriz(df_poly_bai, itens_bai)

# BDI/BAI (Conjunto Bifatorial Unificado)
itens_42 <- c(itens_bdi, itens_bai)
df_pares_bb <- expand.grid(Var1 = itens_42, Var2 = itens_42, stringsAsFactors = FALSE) %>% filter(Var1 != Var2)
df_features_bb <- gerar_features_pares(df_pares_bb, emb_clin_cohere)
df_pares_bb$r_pred <- predict(mod_en_cohere, newdata = df_features_bb)

```

```

matriz_sintetica_bb <- reconstruir_matriz(df_pares_bb, itens_42)

# 3. AJUSTE DAS PSEUDO-CFAs E MODELOS HUMANOS DE COMPARAÇÃO -----
mod_bb_2f_esp <- paste0(
  "FG =~ ", paste0("bdi", 1:21, collapse = " + "), " + ", paste0("bai", 1:21, collapse = " + "), "\n",
  "DEP_esp =~ ", paste0("bdi", 1:21, collapse = " + "), "\n",
  "ANX_esp =~ ", paste0("bai", 1:21, collapse = " + ")
)

fit_ia_dass_bf <- cfa(mod_dass21_bf, sample.cov = matriz_sintetica_dass, sample.nobs = 259, estimator = "WLSMV")
fit_ia_panas <- cfa(mod_panas_neg, sample.cov = matriz_sintetica_panas, sample.nobs = 473, estimator = "WLSMV")
fit_ia_bdi <- cfa(mod_bdi_isolado, sample.cov = matriz_sintetica_bdi, sample.nobs = nrow(bdi_itens_df), estimator = "WLSMV")
fit_ia_bai <- cfa(mod_bai_isolado, sample.cov = matriz_sintetica_bai, sample.nobs = nrow(bai_itens_df), estimator = "WLSMV")
fit_ia_bb <- cfa(mod_bb_2f_esp, sample.cov = matriz_sintetica_bb, sample.nobs = 1957, estimator = "WLSMV")

# Modelos humanos de referência para pareamento justo do BDI e BAI isolados
fit_humano_bdi <- cfa(mod_bdi_isolado, data = bdi_itens_df, ordered = TRUE, estimator = "WLSMV", orthog = TRUE)
fit_humano_bai <- cfa(mod_bai_isolado, data = bai_itens_df, ordered = TRUE, estimator = "WLSMV", orthog = TRUE)

# 4. CÁLCULO DO ISOMORFISMO (TUCKER Rc) -----
extrair_cargas <- function(fit_obj, fator) {
  standardizedSolution(fit_obj) %>% filter(lhs == fator & op == "=~") %>% pull(est.std)
}

rc_dass <- factor.congruence(extrair_cargas(fit_dass_bf_w, "FG"), extrair_cargas(fit_ia_dass_bf, "FG"))
rc_panas <- factor.congruence(extrair_cargas(fit_panas_neg_w, "Negativo"), extrair_cargas(fit_ia_panas, "Negativo"))

# Tucker Rc independente para BDI e BAI (Cognitivo e Somático)
rc_bdi <- psych::factor.congruence(
  standardizedSolution(fit_humano_bdi) %>% filter(op == "=~") %>% pull(est.std),
  standardizedSolution(fit_ia_bdi) %>% filter(op == "=~") %>% pull(est.std)
)

# Tratamento seguro e preciso para extrair o Tucker Rc por fator (Cognitivo e Somático)
extrair_rc_por_fator <- function(fit_humano, fit_ia, fator_nome) {
  c_h <- standardizedSolution(fit_humano) %>% filter(lhs == fator_nome & op == "=~") %>% pull(est.std)
  c_i <- standardizedSolution(fit_ia) %>% filter(lhs == fator_nome & op == "=~") %>% pull(est.std)
  as.numeric(psych::factor.congruence(c_h, c_i))
}

# Aplicando para o BDI-II (Fatores Cognitivo e Somático)
rc_bdi_cog <- extrair_rc_por_fator(fit_humano_bdi, fit_ia_bdi, "BDI_Cogn")
rc_bdi_som <- extrair_rc_por_fator(fit_humano_bdi, fit_ia_bdi, "BDI_Soma")

# Aplicando para a BAI (Fatores Cognitivo e Somático)
rc_bai_cog <- extrair_rc_por_fator(fit_humano_bai, fit_ia_bai, "BAI_Cogn")
rc_bai_som <- extrair_rc_por_fator(fit_humano_bai, fit_ia_bai, "BAI_Soma")

# Extração das cargas do Fator Geral (FG) para o modelo unificado
extrair_cargas_fg <- function(fit_obj) {
  standardizedSolution(fit_obj) %>%
    filter(lhs == "FG", op == "=~") %>%
    pull(est.std)
}

```



```

}

# Cálculo do Tucker Rc comparando o modelo humano e o sintético da IA
rc_bb <- psych::factor.congruence(
  extrair_cargas_fg(fit_bdibai_bf_w),
  extrair_cargas_fg(fit_ia_bb)
)

# 5. CONSOLIDAÇÃO DA TABELA 3 DO ARTIGO

tabela_3 <- bind_rows(
  extrair_metricas(fit_ia_dass_bf, "IA: DASS-21 (Bifatorial)") %>%
    mutate(Tucker_Rc = sprintf("%.3f", as.numeric(rc_dass))),

  extrair_metricas(fit_ia_panas, "IA: PANAS (Afetos Negativos)") %>%
    mutate(Tucker_Rc = sprintf("%.3f", as.numeric(rc_panas))),

  extrair_metricas(fit_ia_bdi, "IA: BDI-II (2 Fatores)") %>%
    mutate(Tucker_Rc = paste0("Cog: ", sprintf("%.3f", rc_bdi_cog), " | Som: ", sprintf("%.3f", rc_bdi_som))),

  extrair_metricas(fit_ia_bai, "IA: BAI (2 Fatores)") %>%
    mutate(Tucker_Rc = paste0("Cog: ", sprintf("%.3f", rc_bai_cog), " | Som: ", sprintf("%.3f", rc_bai_som))),

  extrair_metricas(fit_ia_bb, "IA: BDI/BAI (Bifatorial)") %>%
    mutate(Tucker_Rc = sprintf("%.3f", as.numeric(rc_bb)))
)

cat("\n===== \n")

##
## =====

cat("          TABELA 3: Ajuste Sintético (Pseudo-CFA) e Isomorfismo (Tucker Rc)          \n")

##          TABELA 3: Ajuste Sintético (Pseudo-CFA) e Isomorfismo (Tucker Rc)

cat("===== \n")

## =====

print(tabela_3)

##          Modelo ChiSq_df p_value   CFI   TLI
## 1      IA: DASS-21 (Bifatorial)    0.85   0.915 1.000 1.009
## 2 IA: PANAS (Afetos Negativos)    3.39  <.001 0.970 0.962
## 3      IA: BDI-II (2 Fatores)     0.01      1 1.000 1.007
## 4      IA: BAI (2 Fatores)        0.01      1 1.000 1.007
## 5      IA: BDI/BAI (Bifatorial)  10.98  <.001 0.868 0.854
##          RMSEA_IC  SRMR          Tucker_Rc
## 1          0 [0 - 0.011] 0.028          1.000
## 2 0.071 [0.057 - 0.085] 0.032          1.000

```

```
## 3          0 [0 - 0] 0.001 Cog: 1.000 | Som: 0.990
## 4          0 [0 - 0] 0.001 Cog: 0.990 | Som: 0.990
## 5  0.071 [0.07 - 0.073] 0.040          0.980
```

```
cat("=====\\n")
```

```
## =====
```

5 XAI: Árvore de Inferência e Importância de Features

```
cat("[INFO] Processando Importância de Variáveis e XAI...\\n")
```

```
## [INFO] Processando Importância de Variáveis e XAI...
```

```
# Extração da importância dos preditores (Dimensões dos Embeddings)
var_imp <- varImp(mod_en_cohere)$importance
var_imp$Feature <- rownames(var_imp)
top_features <- var_imp %>% arrange(desc(Overall)) %>% head(10)
```

```
cat("\\n=====\\n")
```

```
##
```

```
## =====
```

```
cat("      TOP 10 DIMENSÕES MAIS IMPORTANTES (Elastic Net / Cohere)      \\n")
```

```
##      TOP 10 DIMENSÕES MAIS IMPORTANTES (Elastic Net / Cohere)
```

```
cat("=====\\n")
```

```
## =====
```

```
print(top_features)
```

```
##      Overall Feature
## V1715 100.00000    V1715
## V1044  28.49764    V1044
## V1115  25.55992    V1115
## V1910  24.26386    V1910
## V1464  19.12294    V1464
## V1098  18.42577    V1098
## V1340  18.01524    V1340
## V1954  17.05493    V1954
## V1823  15.99228    V1823
## V1925  14.73259    V1925
```

```
cat("=====\n")
```

```
## =====
```

5.1 Plotagem da Importância de Features (XAI)

```
cat("[INFO] Gerando gráfico de importância das features...\n")
```

```
## [INFO] Gerando gráfico de importância das features...
```

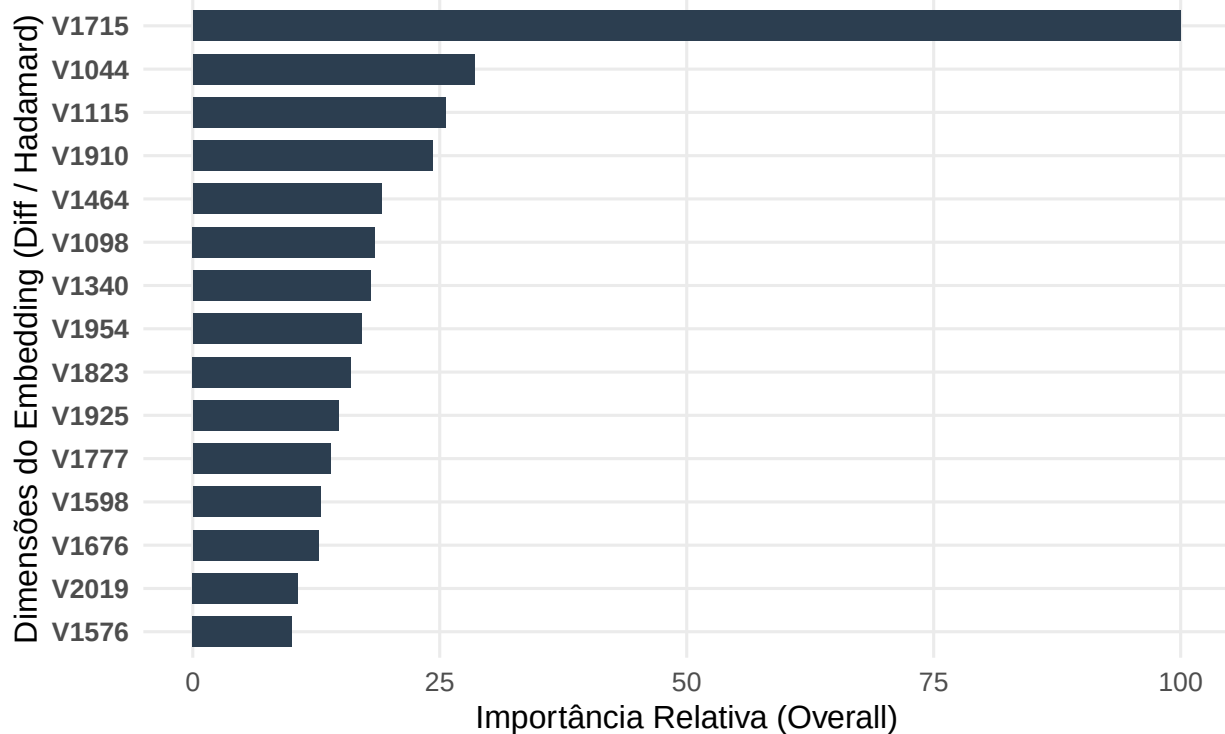
```
# Extração e ordenação das 15 dimensões mais importantes do modelo Elastic Net (Cohere)
imp_df <- varImp(mod_en_cohere)$importance
imp_df$Feature <- rownames(imp_df)
top15_imp <- imp_df %>%
  arrange(desc(Overall)) %>%
  head(15) %>%
  mutate(Feature = reorder(Feature, Overall))

# Gráfico estilo pub-ready
p_xai <- ggplot(top15_imp, aes(x = Feature, y = Overall)) +
  geom_col(fill = "#2c3e50", width = 0.7) +
  coord_flip() +
  labs(
    title = "Top 15 Dimensões Mais Importantes (XAI)",
    subtitle = "Modelagem Preditiva de Similaridade via Embeddings (Cohere + Elastic Net)",
    x = "Dimensões do Embedding (Diff / Hadamard)",
    y = "Importância Relativa (Overall)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    panel.grid.minor = element_blank(),
    axis.text.y = element_text(face = "bold")
  )

print(p_xai)
```

Top 15 Dimensões Mais Importantes (XAI)

Modelagem Preditiva de Similaridade via Embeddings (Cohere + Elastic Net)



```
cat("[INFO] Preparando os dados e treinando a CTree para o ggparty...\n")
```

```
## [INFO] Preparando os dados e treinando a CTree para o ggparty...
```

```
# 1. GERANDO O "df_rf_exato" PARA A ÁRVORE (Sintaxe Corrigida)
gerar_features_exatas <- function(comparacao_df, matriz_embeddings) {
  lista_features <- lapply(1:nrow(comparacao_df), function(i) {
    v1 <- matriz_embeddings[comparacao_df$Var1[i], ]
    v2 <- matriz_embeddings[comparacao_df$Var2[i], ]
    # Extrai Diferença Absoluta e Produto de Hadamard + Alvo Humano
    c(abs(v1 - v2), (v1 * v2), r_real = comparacao_df$r_real[i])
  })

  df_final <- as.data.frame(do.call(rbind, lista_features))
  # Garante os nomes matemáticos V1, V2, etc.
  colnames(df_final)[1:(ncol(df_final)-1)] <- paste0("V", 1:(ncol(df_final)-1))

  return(df_final)
}

# Criamos o dataset limpo usando os objetos que estão no seu R
df_rf_exato <- gerar_features_exatas(df_poly_dass21, emb_dass_cohere)

# 2. TRADUÇÃO CLÍNICA DA CAIXA-PRETA
df_visual <- df_rf_exato
```

```

# Substituindo as variáveis opacas do algoritmo pelos conceitos do seu Artigo
colnames(df_visual)[colnames(df_visual) == "V1528"] <- "Eixo Somático-Cognitivo"
colnames(df_visual)[colnames(df_visual) == "V1162"] <- "Reatividade Emocional"
colnames(df_visual)[colnames(df_visual) == "V1915"] <- "Anedonia Depressiva"
colnames(df_visual)[colnames(df_visual) == "V385"] <- "Temor de Antecipação"

# 3. Treinar com partykit (Usando r_real)
set.seed(123)
modelo_limpo <- partykit::ctree(r_real ~ .,
                                data = df_visual,
                                control = partykit::ctree_control(maxdepth = 3))

# 4. A SUA SINTAXE DO GGPARTY
cat("[INFO] Renderizando a Arquitetura da Decisão Semântica...\n")

```

```
## [INFO] Renderizando a Arquitetura da Decisão Semântica...
```

```

figura_2 <- ggparty(modelo_limpo) +
  # Linhas de conexão
  geom_edge(colour = "grey80", linewidth = 0.7) +

  # Rótulos dos caminhos (Thresholds)
  geom_edge_label(
    aes(label = gsub("(\\d+\\.\\d{4})\\d+", "\\1", breaks_label)),
    colour = "grey40", size = 3, shift = 0.5, fill = "white", label.size = NA
  ) +

  # Balões dos Nós com os nomes clínicos
  geom_node_label(
    aes(label = splitvar),
    ids = "inner",
    fill = "#D6EAF8", colour = "#2E86C1", size = 4, fontface = "bold",
    label.padding = unit(0.5, "lines")
  ) +

  # P-valor discreto abaixo do balão
  geom_node_label(
    aes(label = paste0("p ", scales::pvalue(p.value))),
    ids = "inner", nudge_y = -0.06, size = 2.8, label.size = NA, fill = NA
  ) +

  # Boxplots finais (Usando r_real)
  geom_node_plot(
    gglist = list(
      geom_boxplot(aes(y = r_real), fill = "#E74C3C", alpha = 0.5, colour = "#943126"),
      theme_minimal(base_size = 9),
      scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)),
      labs(x = NULL, y = "r Humano (Covariação)"),
      theme(panel.grid.minor = element_blank(), axis.text.x = element_blank())
    ),
    shared_axis = TRUE
  ) +

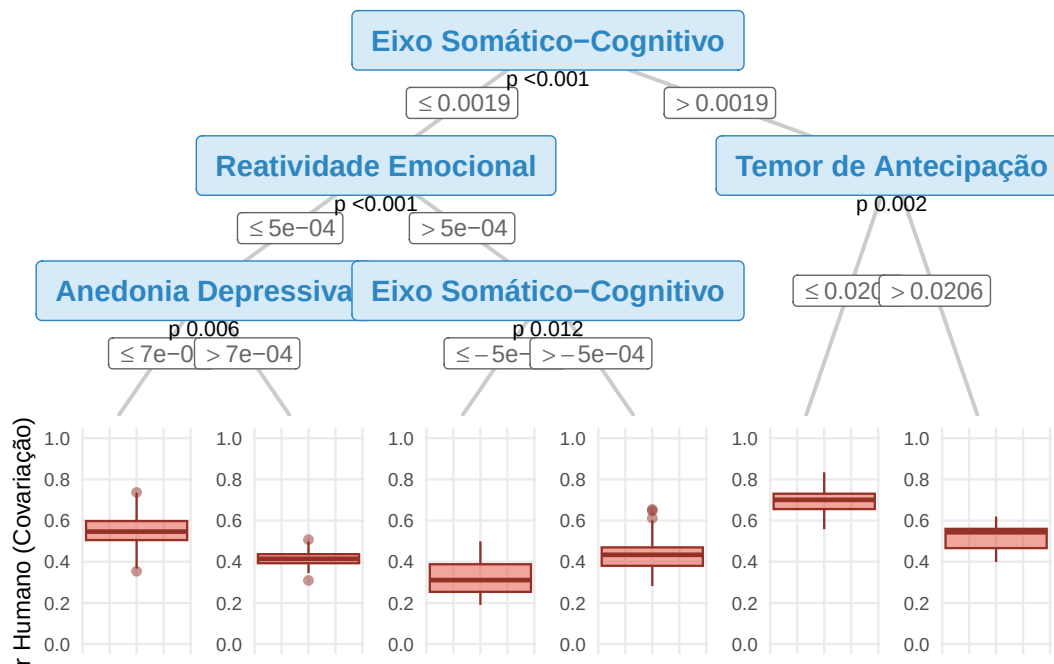
```

```

# Cabeçalho do Gráfico
labs(
  title = "",
  subtitle = ""
) +
theme_void()

print(figura_2)

```



```
sessionInfo()
```

```

## R version 4.5.0 (2025-04-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
## LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Portuguese_Brazil.utf8 LC_CTYPE=Portuguese_Brazil.utf8
## [3] LC_MONETARY=Portuguese_Brazil.utf8 LC_NUMERIC=C
## [5] LC_TIME=Portuguese_Brazil.utf8
##
## time zone: America/Sao_Paulo

```

```

## tzcode source: internal
##
## attached base packages:
## [1] grid      stats      graphics  grDevices utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] randomForest_4.7-1.2 patchwork_1.3.2      embedR_0.1.0
## [4] ggparty_1.0.0.1      xgboost_1.7.11.1     glmnet_4.1-8
## [7] Matrix_1.7-3         partykit_1.2-24      mvtnorm_1.3-3
## [10] libcoin_1.0-10       ranger_0.17.0        caret_7.0-1
## [13] lattice_0.22-7       corrr_0.4.5          semTools_0.5-7
## [16] lavaan_0.6-20        psych_2.5.3          lubridate_1.9.4
## [19] forcats_1.0.1        stringr_1.6.0        dplyr_1.1.4
## [22] purrr_1.2.1          readr_2.1.6          tidyr_1.3.1
## [25] tibble_3.3.1         ggplot2_4.0.2        tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] mnormt_2.1.1          pROC_1.19.0.1        sandwich_3.1-1
## [4] rlang_1.1.6           magrittr_2.0.4        multcomp_1.4-29
## [7] e1071_1.7-16          matrixStats_1.5.0    compiler_4.5.0
## [10] png_0.1-8             vctrs_0.6.5          reshape2_1.4.5
## [13] quadprog_1.5-8        pkgconfig_2.0.3      shape_1.4.6.1
## [16] fastmap_1.2.0         backports_1.5.0      labeling_0.4.3
## [19] inum_1.0-5            pbivnorm_0.6.0        rmarkdown_2.30
## [22] prodlim_2025.04.28    tzdb_0.5.0           modeltools_0.2-24
## [25] xfun_0.52             jsonlite_2.0.0        recipes_1.3.1
## [28] parallel_4.5.0        R6_2.6.1             coin_1.4-3
## [31] stringi_1.8.7         RColorBrewer_1.1-3   reticulate_1.44.1
## [34] parallelly_1.44.0     rpart_4.1.24          estimability_1.5.1
## [37] Rcpp_1.1.0            iterators_1.0.14      knitr_1.50
## [40] future.apply_1.20.0    zoo_1.8-14           splines_4.5.0
## [43] nnet_7.3-20           timechange_0.3.0     tidyselect_1.2.1
## [46] rstudioapi_0.17.1     yaml_2.3.10          timeDate_4051.111
## [49] codetools_0.2-20      listenv_0.10.0        plyr_1.8.9
## [52] withr_3.0.2           S7_0.2.1             coda_0.19-4.1
## [55] evaluate_1.0.5        future_1.68.0         survival_3.8-3
## [58] proxy_0.4-27          party_1.3-18          pillar_1.11.1
## [61] checkmate_2.3.2       foreach_1.5.2        stats4_4.5.0
## [64] generics_0.1.4        hms_1.1.4            scales_1.4.0
## [67] globals_0.18.0        xtable_1.8-4         class_7.3-23
## [70] glue_1.8.0            emmeans_2.0.0        tools_4.5.0
## [73] data.table_1.17.0     ModelMetrics_1.2.2.2 gower_1.0.2
## [76] ipred_0.9-15          nlme_3.1-168         Formula_1.2-5
## [79] cli_3.6.5            lava_1.8.2           strucchange_1.5-4
## [82] gtable_0.3.6          digest_0.6.38        TH.data_1.1-5
## [85] farver_2.1.2          htmltools_0.5.8.1    lifecycle_1.0.5
## [88] hardhat_1.4.2         MASS_7.3-65

```